
Working with Files and Directories

Getting Information on Files

Functions	Usage
<code>file_exists(\$filepath)</code>	checks whether a file or directory exists or not.
<code>filesize(\$filepath)</code>	return the size of a specified file in bytes, or false upon error
<code>fileatime()</code>	returns the time at which the file was last accessed as a UNIX timestamp.
<code>filectime()</code>	returns the last time a file was changed.
<code>filemtime()</code>	returns the last time the file content was modified.
<code>basename()</code>	retrieves the file name from the path.(retrieves the last whole string after the rightmost slash.)

```
<?php
$fileName="file/example.txt";
if(file_exists($fileName)){
    echo "File is there"."<br>";
    echo "Size of the file: ".filesize($fileName)."<br>";
    echo "Last access: ".filemtime($fileName)."<br>";
    echo "Last access: ".date("F d Y H:i:s.", filemtime($fileName));
    echo "Last change: ".filectime($fileName)."<br>";
    echo "Last modify: ".filemtime($fileName)."<br>";
    echo "Basename: ".basename($fileName)."<br>";
}
else{
    echo "File is not there"."<br>";
}
?>
```

Opening a File

- A file handle is a pointer associated with the open file that you can then use to access the file 's contents.
- The **fopen()** function opens a file and returns a file handle associated with the file.
- The first argument specifies the name of the file you want to open, and the second argument specifies the mode , or how the file is to be used.

Syntax: `fopen(filename, mode)`

Eg: \$handle = fopen("./data.txt", "r");

Mode arguments of the fopen() function

Argument	Description
r	Open the file for reading only. The file pointer is placed at the beginning of the file.
r+	Open the file for reading and writing. The file pointer is placed at the beginning of the file.
w	Open the file for writing only. Any existing content will be lost. If the file does not exist, PHP attempts to create it.
w+	Open the file for reading and writing. Any existing file content will be lost. If the file does not exist, PHP attempts to create it.
a	Open the file for appending only. Data is written to the end of an existing file. If the file does not exist, PHP attempts to create it.
a+	Open the file for reading and appending. Data is written to the end of an existing file. If the file does not exist, PHP attempts to create it.
x	Create and open the specified file for writing only; returns false if the file already exists.
x+	Create and open the specified file for reading and writing; returns false if the file already exists.

Closing a File

Once you've finished working with a file, it needs to be closed. We can do this using **fclose()** , passing in the open file ' s handle as a single argument, like this:

Syntax: `fclose($handle)`

Reading and Writing to Files

`fread()` : Reads a string of characters from a file

`fwrite()` : Writes a string of characters to a file

`fgetc()` : Reads a single character at a time

`feof()` : Checks to see if the end of the file has been reached

`fgets()` : Reads a single line at a time

`fgetcsv()` : Reads a line of comma - separated values

`file()` : Reads an entire file into an array

`file_get_contents()` : Reads an entire file into a string without needing to open it

`file_put_contents()` : Writes a whole string to a file without needing to open it

`fpassthru()` : Displays the contents of an open file

`readfile()` : Displays the contents of a file without needing to open it

`fseek()` : Moves the file pointer to a specific location within an open file

`ftell()` : Returns the position of the file pointer

`rewind()` : Moves the file pointer to the start of the file

Reading Strings of Characters

- The **fread()** function can be used to read a string of characters from a file. It takes two arguments: a file handle and the number of characters to read.

```
$handle = fopen( "data.txt", "r");  
$data = fread( $handle, 10 );
```

- The **fgetc()** function can be used to read only one character at a time. It takes a single argument: a file handle.

```
$one_char = fgetc( $handle );
```


Writing Strings of Characters

- The **fwrite()** function to write data to a file. It requires two arguments: a file handle and a string to write to the file.

```
$handle2 = fopen( "data2.txt", "w");  
fwrite($handle2,"Web Technologies");
```

- You can limit the number of characters written by specifying an integer as a third argument

```
fwrite($handle2,"University of Jaffna",5);
```

Testing for the End of a File

- The **feof()** function checks if the "end-of-file" (EOF) has been reached for an open file. It takes a single argument: a file handle.
- It returns true(1) when the file pointer has reached the end of the file (or if an error occurs) and returns false(0) otherwise.

```
echo feof($handle);  
  
while ( !feof( $handle ) ) {  
    $text = fread( $handle, 3 );  
}
```

Reading One Line at a Time

- The **fgets()** function to read text from a file one line at a time. It takes a single argument: a file handle.
- The function reads from the current file pointer to the end of the current line, and returns the read characters as a string.
- You can limit the number of characters read by passing in a second, integer argument

`fgets($handle);`

Reading CSV Files

- The **fgetcsv()** function to read CSV files.

```
/*  
people.csv contains:  
"John","Smith",45  
"Mary","Clark",34  
"Bill","Jim",32  
*/  
  
$handle = fopen( "people.csv", "r:" );  
while ( $record = fgetcsv( $handle ) ) {  
    echo "Full Name is ".$record[0]. " ".$record[1]. " And Age is ".$record[2]. "<br>";  
}  
  
fclose( $handle );
```

Reading and Writing Entire Files

`file()` : For reading a whole file into an array, without needing to open it

`file_get_contents()` and `file_put_contents()` : For reading and writing the contents of a file without needing to open it

`fpassthru()` : For displaying the contents of an open file

`readfile()` : For displaying the contents of a file without needing to open it

Reading and Writing Entire Files

- *file()* reads the contents of a file into an array, with each element containing a line from the file.

```
$lines = file("data.txt" );  
foreach ( $lines as $line ){  
    echo $line . "<br>" ;  
}
```

Reading and Writing Entire Files

- *file_get_contents()* does a similar job to *file()* , but it returns the file contents as a single string, rather than an array of lines.
- Syntax: `file_get_contents(filename,FILE_USE_INCLUDE_PATH flag,context,start,length)`
- All the parameters than filename are optional.

```
$fileContents = file_get_contents( "data.txt" );  
echo $fileContents;  
$fileContents = file_get_contents( "data.txt" ,null,null,10,5);  
echo $fileContents;
```

Reading and Writing Entire Files

- *file_put_contents()* takes a string and writes it to a file:
\$numChars = file_put_contents("data.txt","hello welcome);
- You can affect the behavior of the function by passing various flags as the third argument. *file_put_contents()* supports the same flags as *file_get_contents()* , as well as two additional flags:

Flag	Description
FILE_APPEND	If the file already exists, append the string to the end of the file, rather than overwriting the file.
LOCK_EX	Lock the file before writing to it. This ensures that other processes can't write to the file at the same time.

Reading and Writing Entire Files

- *fpasssthru()* and *readfile()* both take a file and output its unmodified contents straight to the Web browser.
- *fpasssthru()* requires the handle of an open file to work with:
 \$numChars = fpasssthru(\$handle);
- *readfile()* instead works on an unopened file:
 \$numChars = readfile("myfile.txt");

Random Access to File Data

It's possible to move the file pointer around within an open file, so that you can start reading or writing at any point in the file.

PHP gives you three functions that let you work with the file pointer:

`fseek()` : Repositions the file pointer to a specified point in the file

`rewind()` : Moves the file pointer to the start of the file

`ftell()` : Returns the current position of the file pointer

Random Access to File Data

```
<?php
$handle = fopen( "data.txt","r" );
fseek( $handle, 7 );
echo fread( $handle, 5 )."<br>";
echo ftell( $handle )."<br>";
rewind( $handle );
echo ftell( $handle )."<br>";
fseek( $handle, 7 );
echo ftell( $handle )."<br>";
fseek( $handle, 0 );
echo ftell( $handle )."<br>";
fclose( $handle );
?>
```

Working with File Permissions

- Files and directories have three levels of access:
 - User
 - Group
 - Other
- The three typical permissions for files and directories are:
 - Read (r)
 - Write (w)
 - Execute (x)

Working with File Permissions

- File Permissions are calculated using a four digit octal value.
The first digit is always 0.
To assign more than one value to an access level, add the values of the permissions together.

Permissions	First Digit (Leftmost) Always 0	Second Digit User (u)	Third Digit Group (g)	Fourth Digit Other (o)
Read (r)	0	4	4	4
Write (w)	0	2	2	2
Execute (x)	0	1	1	1

Changing Permissions

The `chmod()` function is used to change the permissions or modes of a file or directory.

Syntax : `chmod(filename,mode)`

Eg: `chmod("data.txt",0644);`

Checking File Permissions

- We can check the file permissions using *is_readable()* , *is_writable()* and *is_executable()* functions passing the file name. These will return true or false.
- We can also use the *fileperms()* function to return an integer representing the permissions that are set on a file or directory.

Checking File Permissions

```
<?php
chmod("data.txt",0644);
if ( is_readable( "data.txt")) {
    echo "I can read data.txt"."<br>";
}
if ( is_writable( "data.txt") ){
    echo "I can write data.txt"."<br>";
}
if ( is_executable( "data.txt")) {
    echo "I can execute data.txt"."<br>";
}
echo substr( sprintf( "%o", fileperms( "data.txt" ) ), -4 ); //print the octal value of the permissions
?>
```


Copying, Renaming, and Deleting Files

PHP also lets you copy, rename, and delete files. The functions to perform these operations are *copy()*, *rename()* and *unlink()*, respectively.

Syntax: `copy(from_file_path, to_file_path)`

`rename(old_name, new_name)`

`unlink(file_name)`

Eg: `copy("data.txt", "../folder/hello.txt");`

`rename("data.txt", "eg.txt");`

`unlink("eg.txt");`

Working with Directories

- A directory handle is similar to a file handle; it's a special variable pointing to a directory, which you can obtain via the *opendir()* function:
 `$handle = opendir( "folder" );`
- We can close a directory by passing the directory handle to the function *closedir()*
 `closedir($handle)`
- The *readdir()* function expects a directory handle for an opened directory, and returns the filename of the next entry in the directory:
 `$filename = readdir( $handle );`

Some Other Directory Functions

`rewinddir()` : Moves the directory pointer back to the start of the list of entries

`chdir()` : Changes the current directory

`getcwd()` : Return the current directory

`mkdir()` : Creates a directory

`rmdir()` : Deletes a directory

`dirname()` : Returns the directory portion of a path

Resetting the Directory Pointer

The **rewinddir()** function resets PHP 's internal pointer back to the first entry in a given directory.

We can do this using **rewinddir()** , passing in the open directory 's handle as a single argument, like this:

```
Syntax:    rewinddir($handle)
```

Changing the Current Directory

The **chdir()** function changes the current directory to a new directory. To use **chdir()**, pass the new directory handle to it:

Syntax: `chdir(new directory)`

chdir() returns true if PHP managed to change to the specified directory, or false if there was an error.

Creating Directories

To create a new directory, call the **mkdir()** function, passing in the path of the directory you want to create:

Syntax: `mkdir(path of directory)`

We can also set permissions for the directory at the time you create it by passing the mode as the second argument.

Syntax: `mkdir(path of directory,mode)`

Deleting Directories

The **rmdir()** function removes a given directory. The directory must be empty, and you need appropriate permissions to remove it.

Syntax: `rmdir(directory)`

Getting the Directory Path

The **dirname()** function returns the directory part of a given path. It complements the **basename()** function, which returns the filename portion of a given path.

```
<?php
$path = "C:/Users/staff/Documents/kesavi/webtech/question/hello.php";
$directoryPath = dirname( $path );
echo $directoryPath."<br>";
$filename = basename( $path );
echo $filename."<br>";
?>
```


Telling a File from a Directory

PHP has two functions to help you test for a file or a directory:

`is_dir()` : Returns true if the given filename refers to a directory

`is_file()` : Returns true if the given filename refers to a regular file

```
$filename = "myfile";
```

```
if ( is_dir( $filename ) ) {
```

```
    echo "$filename is a directory.";
```

```
} elseif ( is_file( $filename ) ) {
```

```
    echo "$filename is a file.";
```

```
} else {
```

```
    echo "$filename is neither a directory nor a file.";
```

```
}
```